

Question 1: Read the Bike Details Dataset into a Pandas DataFrame and Display Its First 10 Rows

Python Code

```
import pandas as pd

# Read the Bike Details dataset

df = pd.read_csv("Bike Details.csv")

# Display the first 10 rows

print("First 10 rows of the Bike Details dataset:")

display(df.head(10))
```

If the Dataset Is Not Already in Google Colab

Use the following code to upload the CSV file:

```
from google.colab import files

import pandas as pd
```

```
# Upload the Bike Details CSV file

uploaded = files.upload()

# Read the uploaded CSV file

df = pd.read_csv("Bike Details.csv")

# Display the first 10 rows

display(df.head(10))
```

Explanation

- `import pandas as pd` → imports the Pandas library.
- `pd.read_csv()` → reads the CSV dataset into a DataFrame.
- `df` → stores the Bike Details dataset.
- `df.head(10)` → displays the **first 10 rows**.
- `display()` → shows the DataFrame in a clean tabular format in Google Colab.

Short Answer

```
df = pd.read_csv("Bike Details.csv")

df.head(10)
```

Note: If your actual file name is different, replace `"Bike Details.csv"` with the exact CSV file name.

Bike Details Dataset – Complete Google Colab Answers

Initial Step: Load the Dataset

Run this cell first.

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Load dataset

df = pd.read_csv("Bike Details.csv")

# Display basic information

print("Dataset Shape:", df.shape)

display(df.head())
```

If the CSV is not uploaded to Colab:

```
from google.colab import files
```

```
uploaded = files.upload()
```

Then run:

```
df = pd.read_csv("Bike Details.csv")
```

```
display(df.head())
```

Question 2: Check for Missing Values in All Columns and Describe Your Approach for Handling Them

Python Code

```
# Check missing values
```

```
missing_values = df.isnull().sum()
```

```
print("Missing Values in Each Column:")
```

```
print(missing_values)
```

```
# Missing value percentage
```

```
missing_percentage = (df.isnull().sum() / len(df)) * 100
```

```
print("\nMissing Value Percentage:")
```

```
print(missing_percentage)
```

Visualize Missing Values

```
plt.figure(figsize=(10, 5))
```

```
sns.heatmap(df.isnull(), cbar=False)
```

```
plt.title("Missing Values Heatmap")
```

```
plt.xlabel("Columns")
```

```
plt.ylabel("Rows")
```

```
plt.show()
```

Approach for Handling Missing Values

The approach depends on the column and the percentage of missing values.

Numerical Columns

For numerical columns such as `selling_price` or `km_driven`, we can use the **median** when appropriate:

```
df["km_driven"] = df["km_driven"].fillna(df["km_driven"].median())
```

Median is often preferred when the data contains outliers because it is less affected by extreme values.

Categorical Columns

For categorical columns such as `seller_type` or `owner`, we can use the **mode**:

```
df["seller_type"] = df["seller_type"].fillna(df["seller_type"].mode()[0])
```

If Missing Values Are Very Few

Rows can sometimes be removed:

```
df = df.dropna()
```

Conclusion

My approach: First identify the number and percentage of missing values. Then use median/mode imputation where appropriate, and remove rows only when the amount of missing data is small or the records are not useful.

Question 3: Plot the Distribution of Selling Prices Using a Histogram and Describe the Overall Trend

Python Code

```
plt.figure(figsize=(10, 6))

plt.hist(

    df["selling_price"].dropna(),

    bins=30,

    edgecolor="black"

)

plt.title("Distribution of Bike Selling Prices")

plt.xlabel("Selling Price")

plt.ylabel("Frequency")

plt.show()
```

Improved Histogram

Because selling prices can have a wide range, a log-scale version can also be useful:

```
plt.figure(figsize=(10, 6))

plt.hist(

    np.log1p(df["selling_price"].dropna()),

    bins=30,

    edgecolor="black"

)

plt.title("Distribution of Log-Transformed Selling Prices")

plt.xlabel("log(1 + Selling Price)")

plt.ylabel("Frequency")

plt.show()
```

Observation

The selling-price distribution should be examined for **skewness and concentration**. In many used-vehicle datasets, prices are concentrated toward the lower-to-middle range, with fewer expensive bikes creating a **right-skewed distribution**.

The exact trend should be confirmed from the histogram of the supplied dataset.

Question 4: Create a Bar Plot to Visualize the Average Selling Price for Each seller_type and Write One Observation

Python Code

```
average_price =  
df.groupby("seller_type")["selling_price"].mean().sort_values(  
  
    ascending=False  
  
)  
  
print("Average Selling Price by Seller Type:")  
  
print(average_price)
```

Bar Plot

```
plt.figure(figsize=(8, 5))  
  
average_price.plot(kind="bar", edgecolor="black")
```

```
plt.title("Average Selling Price by Seller Type")

plt.xlabel("Seller Type")

plt.ylabel("Average Selling Price")

plt.xticks(rotation=0)


plt.show()
```

Observation

The bar plot compares the average selling price across different `seller_type` categories.

Observation: The seller type with the tallest bar has the **highest average selling price**, while the seller type with the shortest bar has the lowest average selling price.

The exact seller type should be identified from the output generated by your dataset.

Question 5: Compute the Average km_driven for Each Ownership Type and Present the Result as a Bar Plot

Python Code

```
average_km = df.groupby("owner")["km_driven"].mean().sort_values(
```

```
        ascending=False

    )

    print("Average KM Driven by Ownership Type:")

    print(average_km)
```

Bar Plot

```
plt.figure(figsize=(10, 6))

average_km.plot(kind="bar", edgecolor="black")

plt.title("Average KM Driven by Ownership Type")

plt.xlabel("Ownership Type")

plt.ylabel("Average KM Driven")

plt.xticks(rotation=45)

plt.show()
```

Observation

The plot shows the average distance travelled by bikes according to their ownership category.

Observation: Ownership categories with a higher average `km_driven` indicate that bikes in those categories have generally been used more extensively.

The exact highest and lowest ownership categories can be obtained from `average_km`.

Question 6: Use the IQR Method to Detect and Remove Outliers from `km_driven`. Show Before-and-After Summary Statistics

Step 1: Before Cleaning

```
print("Before Removing Outliers:")

print(df["km_driven"].describe())
```

Step 2: Calculate IQR

The **Interquartile Range (IQR)** is:

$$\text{IQR} = Q3 - Q1$$

where:

- $Q1$ = 25th percentile
- $Q3$ = 75th percentile

The lower and upper limits are:

$\text{Lower} = Q1 - 1.5 \times \text{IQR}$

$\text{Upper} = Q3 + 1.5 \times \text{IQR}$

Python Code

```
Q1 = df["km_driven"].quantile(0.25)
```

```
Q3 = df["km_driven"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
print("Q1:", Q1)
```

```
print("Q3:", Q3)
```

```
print("IQR:", IQR)
```

```
print("Lower Bound:", lower_bound)
```

```
print("Upper Bound:", upper_bound)
```

Step 3: Detect Outliers

```
outliers = df[
```

```
(df["km_driven"] < lower_bound) |  
  
(df["km_driven"] > upper_bound)  
  
]  
  
print("Number of Outliers:", len(outliers))  
  
display(outliers.head())
```

Step 4: Remove Outliers

```
df_clean = df[  
  
    (df["km_driven"] >= lower_bound) &  
  
    (df["km_driven"] <= upper_bound)  
  
].copy()
```

Step 5: After Cleaning

```
print("After Removing Outliers:")  
  
print(df_clean["km_driven"].describe())
```

Before vs After Summary

```
before = df["km_driven"].describe()

after = df_clean["km_driven"].describe()


comparison = pd.DataFrame({

    "Before": before,

    "After": after

})


display(comparison)
```

Observation

After applying the IQR method, extreme values outside the lower and upper bounds are removed.

The **maximum km_driven and overall spread should decrease** if high-distance outliers were present.

Question 7: Create a Scatter Plot of year vs. selling_price

Python Code

```
plt.figure(figsize=(10, 6))

plt.scatter(

    df["year"],

    df["selling_price"],

    alpha=0.5

)

plt.title("Year vs Selling Price")

plt.xlabel("Year")

plt.ylabel("Selling Price")

plt.show()
```

Add a Trend Line

```
plt.figure(figsize=(10, 6))

sns.regplot(
```

```
data=df,

x="year",

y="selling_price",

scatter_kws={"alpha": 0.4},

line_kws={}

)

plt.title("Year vs Selling Price")

plt.xlabel("Year")

plt.ylabel("Selling Price")

plt.show()
```

Observation

The scatter plot helps examine whether newer bikes tend to have higher selling prices.

A generally **positive relationship** would indicate that newer bikes tend to sell at higher prices, while older bikes tend to have lower prices.

However, the scatter plot may also show considerable variation because selling price can be influenced by other factors such as `km_driven`, `owner`, and `seller_type`.

Question 8: Convert seller_type into Numeric Format Using One-Hot Encoding

One-hot encoding converts categorical values into separate binary columns.

For example:

seller_type

Individual

Dealer

can become:

seller_type_Dealer

seller_type_Individual

Python Code

```
# One-hot encode seller_type
```

```
df_encoded = pd.get_dummies(
```

```
    df,
```

```
        columns=["seller_type"],  
  
        dtype=int  
  
)  
  
# Display first 5 rows  
  
display(df_encoded.head(5))
```

Check New Columns

```
print("Columns after One-Hot Encoding:")  
  
print(df_encoded.columns.tolist())
```

Explanation

If the original column contains:

Individual

Dealer

one-hot encoding creates columns such as:

seller_type_Dealer

seller_type_Individual

A value of 1 indicates that the row belongs to that category, while 0 indicates that it does not.

Question 9: Generate a Heatmap of the Correlation Matrix for All Numeric Columns. What Correlations Stand Out the Most?

Python Code

```
# Select numeric columns

numeric_df = df.select_dtypes(include=np.number)

# Calculate correlation matrix

correlation_matrix = numeric_df.corr()

print("Correlation Matrix:")

display(correlation_matrix)
```

Heatmap

```
plt.figure(figsize=(10, 7))

sns.heatmap(

    correlation_matrix,

    annot=True,

    cmap="coolwarm",

    fmt=".2f",

    linewidths=0.5

)

plt.title("Correlation Heatmap of Numeric Variables")

plt.show()
```

Find Strongest Correlations Automatically

```
corr_pairs = (

    correlation_matrix

    .where(np.triu(np.ones(correlation_matrix.shape), k=1).astype(bool))

    .stack()
```

```
        .sort_values(key=abs, ascending=False)

)

print("Strongest Correlations:")

print(corr_pairs.head(10))
```

Observation

The heatmap shows the strength and direction of relationships between numeric variables.

For this dataset, pay particular attention to:

- year VS. selling_price
- km_driven VS. selling_price
- year VS. km_driven

A positive correlation means the variables tend to increase together, while a negative correlation means one tends to decrease as the other increases.

Important: Correlation does not prove causation.

Question 10: Summarize Your Findings in a Brief Report

Bike Details Dataset – Findings Report

1. Important Factors Affecting Selling Price

Based on the exploratory analysis, the major factors to investigate for their relationship with `selling_price` are:

Year

The `year` variable represents the bike's model year. The scatter plot can show whether newer bikes generally have higher selling prices.

KM Driven

`km_driven` represents the distance travelled by the bike. Higher mileage may be associated with lower selling prices, although the actual relationship should be confirmed using the correlation matrix and plots.

Seller Type

Different `seller_type` categories may have different average selling prices. The bar plot allows us to compare these categories.

Ownership

The `owner` variable provides information about the ownership history of the bike. It can also be examined as a factor associated with selling price.

2. Data Cleaning Performed

The following data-cleaning steps were performed:

Missing Values

Missing values were identified using:

```
df.isnull().sum()
```

Numerical missing values can be handled using median imputation, while categorical missing values can be handled using mode imputation when appropriate.

Outlier Removal

The IQR method was applied to `km_driven`.

$$IQR = Q3 - Q1$$

Values below:

$$Q1 - 1.5 \times IQR$$

or above:

$$Q3 + 1.5 \times IQR$$

were treated as outliers and removed.

3. Feature Engineering Performed

One-Hot Encoding

The categorical `seller_type` variable was converted into numerical binary columns using:

```
pd.get_dummies()
```

This makes the categorical variable suitable for Machine Learning algorithms.

Numeric Feature Analysis

Numeric columns were analyzed using correlation:

```
df.select_dtypes(include=np.number).corr()
```

Final Conclusion

The Bike Details dataset was explored using **data cleaning, visualization, correlation analysis, outlier detection, and categorical encoding**.

The analysis focused on understanding how variables such as:

- `year`
- `km_driven`
- `seller_type`
- `owner`

relate to `selling_price`.

The `year` vs. `selling_price` scatter plot helps investigate the relationship between bike age and price, while the seller-type bar plot compares average prices across seller categories. The IQR method helps reduce the effect of extreme `km_driven` values,

and one-hot encoding converts categorical seller information into a Machine Learning-friendly numerical format.

The exact strongest factors and numerical observations should be taken from the outputs generated after running these cells on the provided Bike Details dataset.

Complete Analysis Workflow

Load Dataset



Check Missing Values



Data Cleaning



Selling Price Distribution



Seller Type Analysis



Ownership & KM Analysis



IQR Outlier Removal

↓

Year vs Selling Price

↓

One-Hot Encoding

↓

Correlation Heatmap

↓

Final Findings & Report

Important Google Colab Note

Run the cells **in order**. For Questions 6 onward, if you want the cleaned dataset to be used in later analysis, you can replace `df` with `df_clean` after the outlier-removal step.

For example:

```
df = df_clean.copy()
```

Then continue with the remaining analysis.
